Mobile Application Development

Lecture 1

Course Teacher: Md. Mushfiqul Haque
Courtesy: Md. Atiqur Rahman

Native vs Cross Platform

Native

- You can use java or kotlin to develop android mobile applications and swift for apple mobile applications.
- But applications developed by java will not work in apple mobile applications since that application is native and same goes for applications developed by swift.

Cross platform

- Now if you use React native or Flutter to develop your mobile application then that application can run in both android and apple platform without any major modifications.

- That’s why React Native or Flutter is cross platform framework.

React vs React Native

- React JS is used to build the user interface of web applications (that is, apps
  that run on a web browser)
- React Native is used to build applications that run on both iOS and Android devices (that is, cross-platform mobile applications)
- React uses HTML, CSS and JavaScript to create interactive user interfaces. React Native, on the other hand, uses native UI components and APIs to create mobile apps.

Similarity between React and React Native

- They both use the same core React library.
- They both use the same component-based architecture, which means that developers can break down their applications into smaller, more manageable pieces.
- They both use JavaScript as their programming language, and JSX as their templating language.
- They both utilize Chrome DevTools for debugging applications.
- They use the same JavaScript APIs.
- Both React and React Native also use the same styling techniques and components, though React Native's is a bit different.

How React is better than classic HTML,JS,CSS

How web browser renders websites

- Web browser ask for website from the web server at the given URL. Web server replies with HTML file.
- Web browser parses HTML file and get the content of the website and links for other style and backend files.
- After parsing browser makes a tree like structure called DOM(Document Object Model). Like background is the root, then we have its leaves like buttons and text fields.
- Now browser request for linked files to apply styling to the content and makes the page interactive by running any linked JS code.
- For example, if the user fills the wrong information in a form, JavaScript can be used to insert a element into the page showing an error message to the user.
- Biggest problem is that we can not reuse an element, we need to create two different elements or buttons to show the error.

```
<button id="normalBtn">Submit</button>
<button id="errorBtn" style="display:none; color:red;">Error</button>
```

Continued…

React can do the same thing with just one element or button.
Another thing is that data, logic and view are separated in React implementation.

```
ction MyButton({ error }) {
eturn (
 <button style={{ color: error ? 'red' : 'black' }}
   {error ? 'Error' : 'Submit'}
 </button>
;



Button error={false} />  // Normal button
Button error={true} />   // Error button
```

React Native vs Flutter

Functionalities	React Native	Flutter
Programming Language	Java Script	Dart
Components Library	Large Inclusive Library	Smaller Non-Inclusive Library
Adaptive Components	Automatically Adaptive	Need to be Configured Manually
Ecosystem	Many Packages Available	Fewer Number of Packages Available

- We choose **React Native** over **Flutter** because it uses **JavaScript**, which is also used in **backend (Node.js)**.
- This allows us to become **full-stack JS developers** without learning a separate language like **Dart**.
- While Firebase or MongoDB can remove the need for backend in small apps, to meet **industrial standards**, we should still learn **backend development** using **Node.js**.

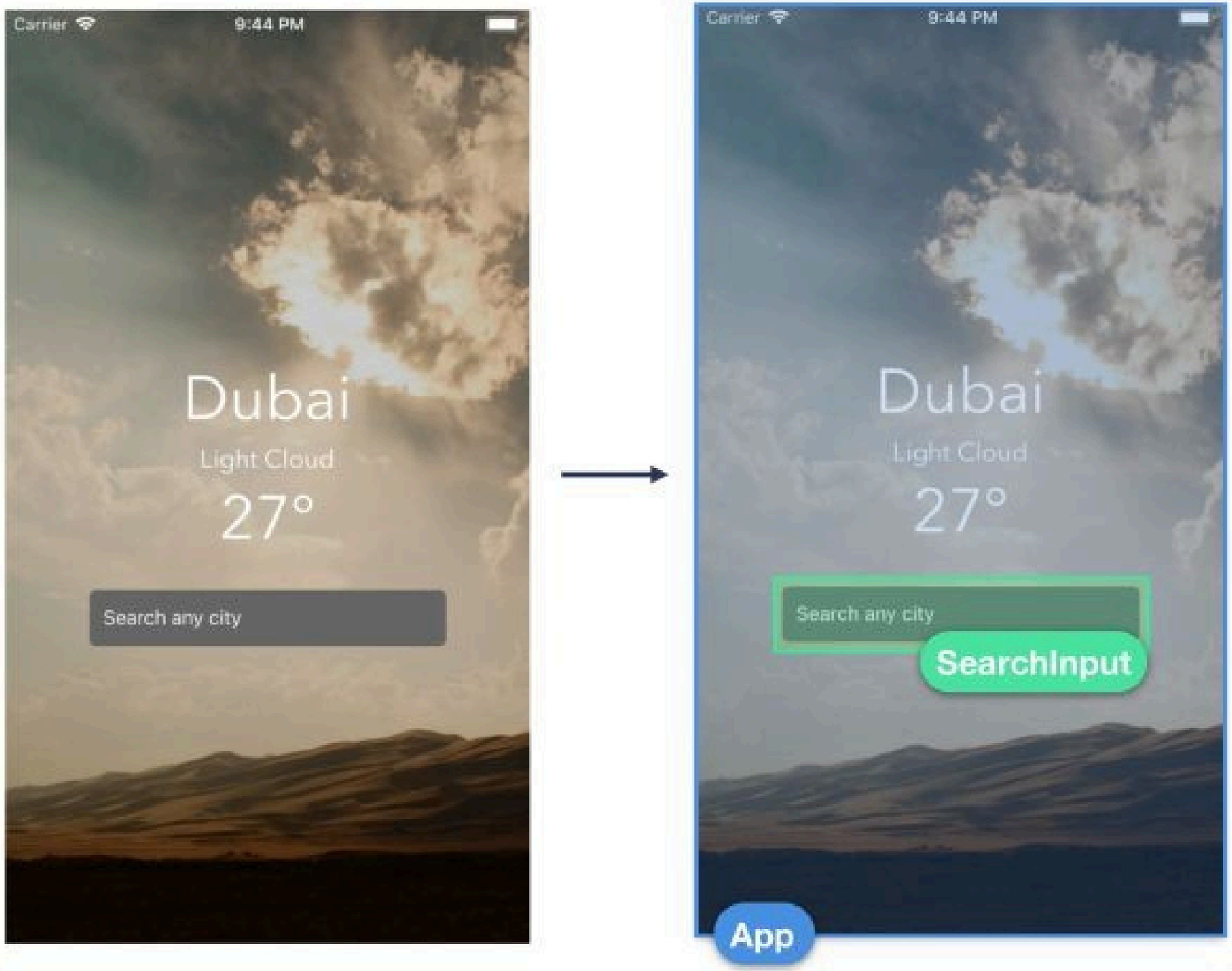
Getting Started With React Native

Things we will cover

- How to use Snack
- Using core and custom components
- Passing data between components
- Handling component state
- Handling user input
- Applying styles to components
- Fetching data from a remote API

Component Structure

To make an application, we need some designs or prototype.
And our first task is to convert the prototype into a component structure.



App.js

```jsx
import { Text, SafeAreaView, StyleSheet } from 'react-native';

// You can import supported modules from npm
import { Card } from 'react-native-paper';

// or any files within the Snack
import AssetExample from './components/AssetExample';

export default function App() {
  return (
    <SafeAreaView style={styles.container}>
      <Text style={styles.paragraph}>
        Change code in the editor and watch it change on your phone! Save to get a shareable url.
      </Text>
      <Card>
        <AssetExample />
      </Card>
    </SafeAreaView>
  );
}

const styles = StyleSheet.create({
  container: {
    flex: 1,
    justifyContent: 'center',
    backgroundColor: '#ecf0f1',
    padding: 8,
  },
  paragraph: {
    margin: 24,
    fontSize: 18,
    fontWeight: 'bold',
    textAlign: 'center',
  },
});
```

Function Structured

weather/1/App.js

```jsx
import React from 'react';
import { StyleSheet, Text, View } from 'react-native';

export default class App extends React.Component {
  render() {
    return (
      <View style={styles.container}>
        <Text>Open up App.js to start working on your app!</Text>
        <Text>Changes you make will automatically reload.</Text>
        <Text>Shake your phone to open the developer menu.</Text>
      </View>
    );
  }
}

const styles = StyleSheet.create({
  container: {
    flex: 1,
    backgroundColor: '#fff',
    alignItems: 'center',
    justifyContent: 'center',
  },
});
```

Class Structured

JSX

- Stands for JavaScript XML.
- Allows us to write HTML elements in JavaScript and place them in DOM without any createElement() and/or appendChild() methods.
- JSX makes it easier to write and add HTML in React.
- The boxed piece of code is JSX.
- The right image has a different depiction of same function structure.

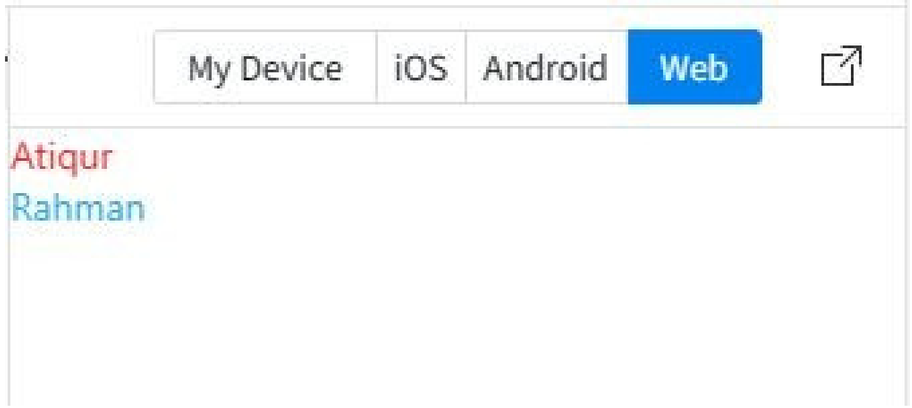
```jsx
import React from 'react';

const MyComponent = () => {
  return (
    <div>
      <h1>Hello, World!</h1>
      <p>This is my first React component.</p>
    </div>
  );
};

export default MyComponent;
```

StyleSheet

- StyleSheet is a module in React Native that allows you to manage styles for your components.
- This module works like CSS and but its not CSS.
- You can directly insert styles using props without using StyleSheet.

```
import { Text, View } from 'react-native';

const App = () => (
  <View>
    <Text style={{color:'#fd2929'}}>Atiqur</Text>
    <Text style={{color : '#29b0fd'}}>Rahman</Text>
  </View>
);

export default App;
```

```
import { Text, View, StyleSheet } from 'react-native';

const App = () => (
  <View>
    <Text style={styles.redText}>Atiqur</Text>
    <Text style={styles.blueText}>Rahman</Text>
  </View>
);

export default App;

const styles = StyleSheet.create({
  redText:{
    color : '#fd2929',
  },
  blueText:{
    color : '#29b0fd',
  }
});
```



Render vs Return

- Class can not return anything but a function can.
- That’s why Class based components has a function render() which returns the JSX (JS which looks like HTML).
- Where function based components just returns the JSX without any intervention.

```
class NewComponent extends React.Component {
    render() {
        return (
            <div>
               <h1>Title</h1>
            </div>
        )
    }
}
```

```
const NewComponent = (props) => {
   return (
            <div>
               <h1>Title</h1>
            </div>
    )
}
```

Props

- Used to pass data from one component to another.
- Similar to arguments and parameters we use in other programming languages.
- This lets you make a single component that is used in many different places in your app, with slightly different properties.

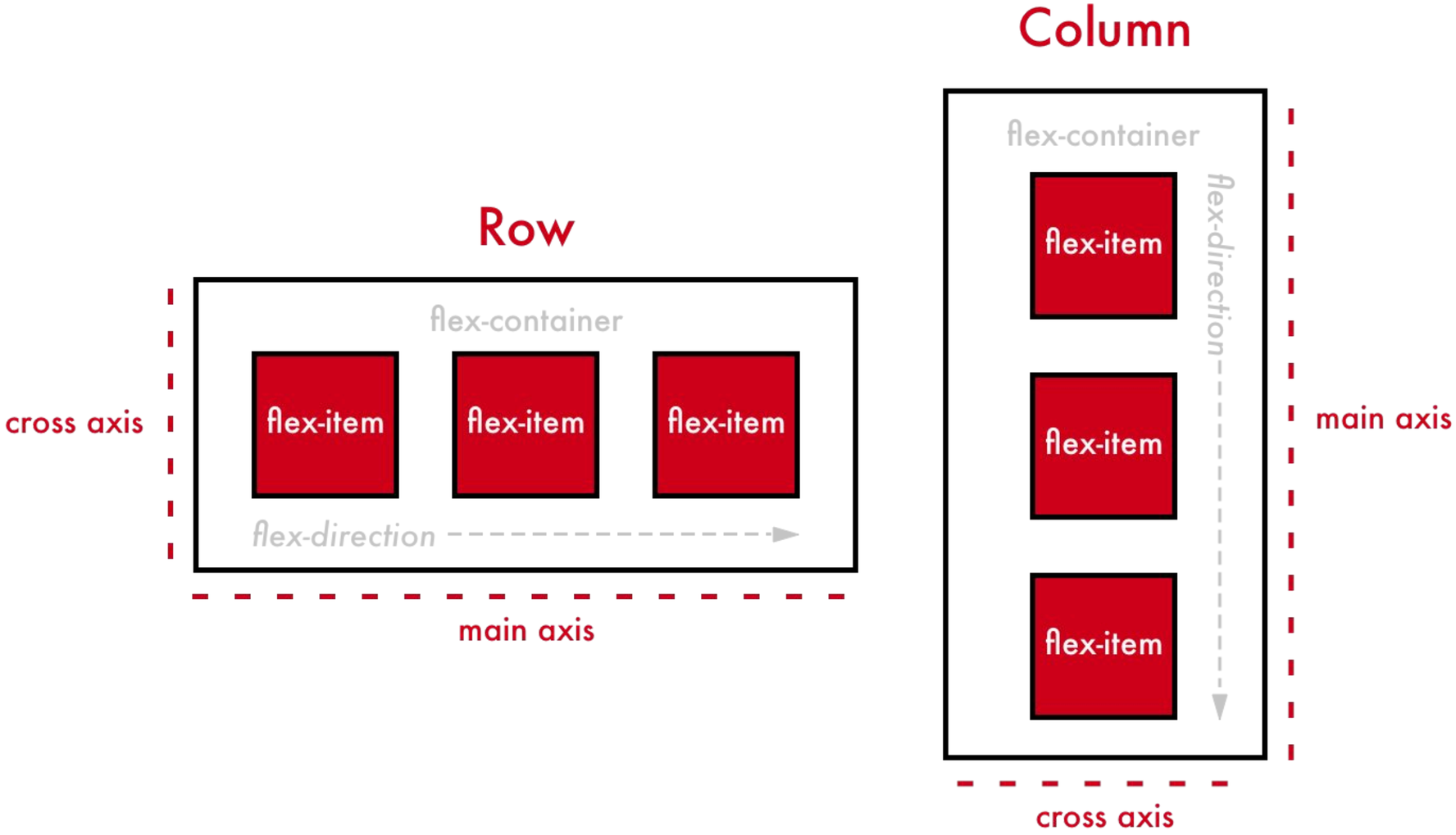
```
import React from 'react';
import { Text, View } from 'react-native';

const Greeting = (props) => {
  return (
    <View style={{ alignItems: 'center' }}>
      <Text>Hello, {props.name}!</Text>
    </View>
  );
};

const App = () => {
  return (
    <View style={{ alignItems: 'center', top: 50 }}>
      <Greeting name="John" />
      <Greeting name="Jane" />
      <Greeting name="Jim" />
    </View>
  );
};

export default App;
```

Basics of Style

- Flex:1 means item will grow or shrink to fill the available space.
- AlignItems applies to cross-axis.
- JustifyContent applies to main-axis.
- If flex-direction is column then main-axis is vertical.
- If flex-direction is row main-axis is horizontal.
- Cross-axis is always opposite of main-axis.

```
import { Text, View,Platform,StyleSheet } from 'react-native';

const App = () => (
  <View style={styles.container}>
    <Text style={[styles.largeText,styles.textStyle]}>San Francisco</Text>
    <Text style={[styles.smallText,styles.textStyle]}>Light Cloud</Text>
    <Text style={[styles.largeText,styles.textStyle]}>24°</Text>
  </View>
);

const styles = StyleSheet.create({
  container: {
    flex: 1,
    backgroundColor: '#fff',
    alignItems: 'center',
    justifyContent: 'center',
  },
  textStyle: {
    textAlign: 'center',
    fontFamily: Platform.OS === 'ios' ? 'AvenirNext-Regular' : 'Roboto',
  },
  largeText: {
    fontSize: 44,
  },
  smallText: {
    fontSize: 18,
  },
});

export default App;
```

Illustration of AlignItems & JustifyContent



Platform Specific Files

- Instead of applying conditional checks using Platform.
- OS a number times throughout the entire component file, we can also leverage the use of platform specific files instead.
- We can create two separate files to represent the same component each with a different extension: .ios.js and .android.js.
- If both files export the same component class name, the React Native packager knows to choose the right file based on the path extension.
- We'll dive deeper into platform specific differences later.

TextInput

- TextInput is a fundamental component in React Native that allows users to input text into an app via a keyboard.
- It has several configurable features, such as auto-correction, auto-capitalization, placeholder text, and different keyboard types, such as a numeric keypad.
- If autoCorrect is true then our input will automatically corrected if there is some spelling mistake or grammatical mistakes.
- PlaceHolder will hold the message when there is no input.
- Margin specifies the space between the component and its parent component.
- Padding specifies the space between the component and its children.
- alignSelf has the same options and effect as alignItems but instead of affecting the children within a container, you can apply this property to a single child to change its alignment within its parent. alignSelf overrides any option

  set by the parent with alignItems.

```
<TextInput
  autoCorrect={false}
  placeholder="Search any city"
  placeholderTextColor="white"
  style={styles.textInput}
/>
```

```
textInput: {
  backgroundColor: '#666',
  color: 'white',
  height: 40,
  width: 300,
  marginTop: 20,
  marginHorizontal: 20,
  paddingHorizontal: 10,
  alignSelf: 'center',
},
```

Custom Components

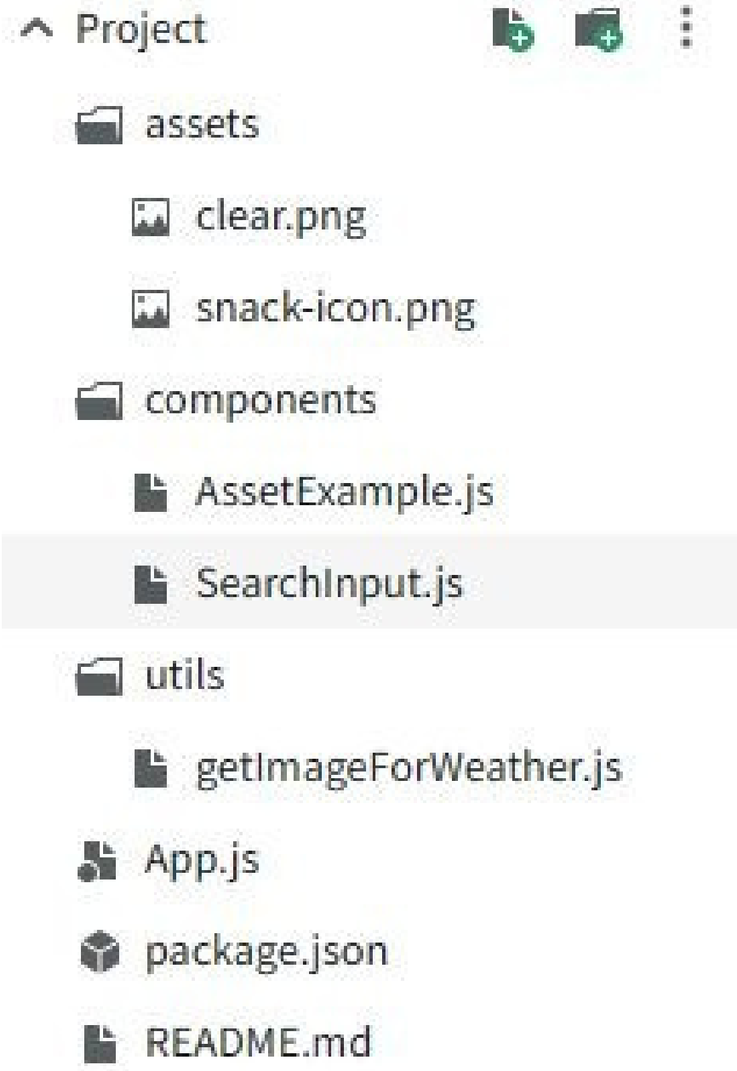
- React Native is component-driven.
- Until now we were coding only in App.js but it's important to begin thinking of how it can further be broken down into smaller and simpler chunks.
- We can do this by creating custom components that contain a small subset of our UI that we feel fits better into a separate, distinct component file.
- This is useful in order to allow us to further split parts of our application into something more manageable, reusable and testable.

```
<View style={styles.container}>
  <Text style={[styles.largeText,styles.textS
  <Text style={[styles.smallText,styles.textS
  <Text style={[styles.largeText,styles.textS

  <SearchInput pHolder="Search Any City"/>

</View>
```

```
const SearchInput = ({pHolder}) => {

  return (

    <View style={styles.container}>

      <TextInput
        autoCorrect={false}
        placeholder={pHolder}
        placeholderTextColor="white"
        style={styles.textInput}
      />

    </View>
  );

}
```

Importing Assets

```
import { Text, View,Platform,StyleSheet,TextInput,ImageBackground } from 'react-native';
import SearchInput from './components/SearchInput'

import getImageForWeather from './utils/getImageForWeather'

const App = () => {

  return (

    <View style={styles.container}>
      <ImageBackground
        source = {getImageForWeather('Clear')}
        style = {styles.imageContainer}
        imageStyle = {styles.image}
      >
        <View style = {styles.detailsContainer}>
          <Text style={[styles.largeText,styles.textStyle]}>San Francisco</Text>
          <Text style={[styles.smallText,styles.textStyle]}>Light Cloud</Text>
          <Text style={[styles.largeText,styles.textStyle]}>24°</Text>

          <SearchInput pHolder="Search Any City"/>
        </View>
      </ImageBackground>
    </View>
  );

}
```



Utility Functions

- Single Line function
- Function returns a map
- Keys are the weather type
- Values are local urls

```
const images = {
  Clear: require('../assets/clear.png'),
  Hail: require('../assets/hail.png'),
  'Heavy Cloud': require('../assets/heavy-cloud.png'),
  'Light Cloud': require('../assets/light-cloud.png'),
  'Heavy Rain': require('../assets/heavy-rain.png'),
  'Light Rain': require('../assets/light-rain.png'),
  Showers: require('../assets/showers.png'),
  Sleet: require('../assets/sleet.png'),
  Snow: require('../assets/snow.png'),
  Thunder: require('../assets/thunder.png'),
};

export default weather => images[weather];
```

Image Component & Style

- Image Background is a default component of React Native
- imageStyle prop is used to add style to the real image
- resize mode property decides how the RAW image should be fit inside its frame (cover, contain, stretch, center, repeat)

```
<ImageBackground
  source = {getImageForWeather('Clear')}
  style = {styles.imageContainer}
  imageStyle = {styles.image}
>
```

```
image: {
  flex:1,
  width:null,
  height:null,
  resizeMode:'cover',
}
```

State

- Now if we type anything inside text input, it will not do anything.
- That's why we need a variable which will be accessible by every element of function component.
- And also if we update that variable, react native engine will re-render the component with the latest value, incorporating real-time update inside our functional component.
- This variable is called State of a functional component.

```
port default class SearchInput extends React.Component
constructor(props) {
  super(props);
  this.state = {
    text: '',
  };
}

handleChangeText = (text) => {
  this.setState({
    text: text,
  });
};

handleSubmitEditing = () => {
  const { onSubmit } = this.props;
  const { text } = this.state;

  if (!text) return;
  onSubmit(text);
  this.setState({
    text: '',
  });
};
```

Communicating With Parent

- We know, parent communicate with its child using props.
- Child communicate with its parent using call back function.
- A callback function is a function passed into another function as an argument, which is then invoked inside the outer function to complete some kind of routine or action.
- Parent gives a function to child, and child use that function to communicate with the parent.
- Communicate means child changes state of the parent to send its message to it.

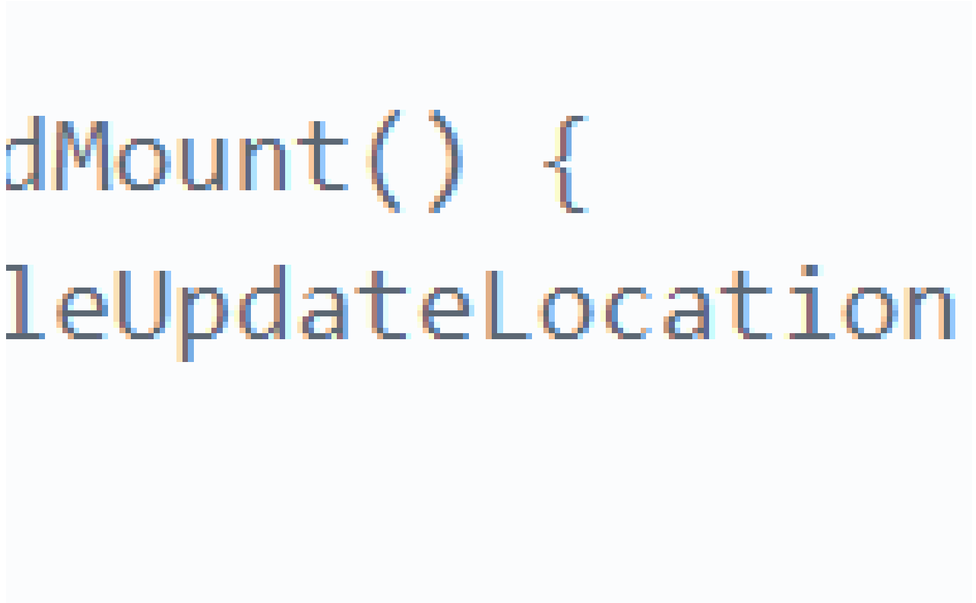
```
<SearchInput
  pHolder="Search Any City"
  onSubmit={handleUpdateLocation}
/>
```

Inside Parent (App)

```
cation = async (city) => {
eturn;

cation, weather, temperature } = await fetchRea
ate({
alse,
: location,
 weather,
ure: temperature,

{
ate({
rue,
```

Lifecycle Methods

Phase	Method
Mounting(**When the component is being created**)	constructor(), render(), componentDidMount()
Updating(**When props or state changes**)	render(), componentDidUpdate()
Unmounting(**When the component is removed from the UI**)	componentWillUnmount()



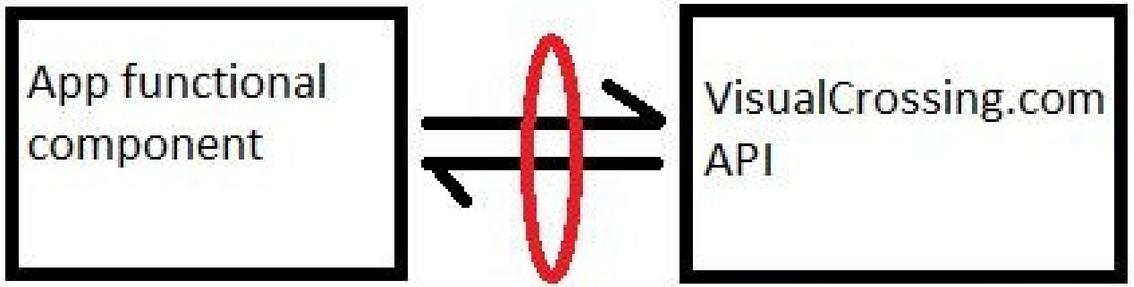
Asynchronous Programming

- This is a technique that enables your program to start a potentially long-running task and still be able to be responsive to other events while that task runs, rather than having to wait until that task has finished. Once that task has finished, your program is presented with the result.
- You will see fetch() function inside api.js which initiates asynchronous programming.
- Whenever we need to wait for a function (fetch) to completes its task we add await before it.
- Whenever we call a await function inside another function, then we need to declare the outer function as async function (fetchRealWeather).
- We might need to introduce a chain of async/await to complete our task.

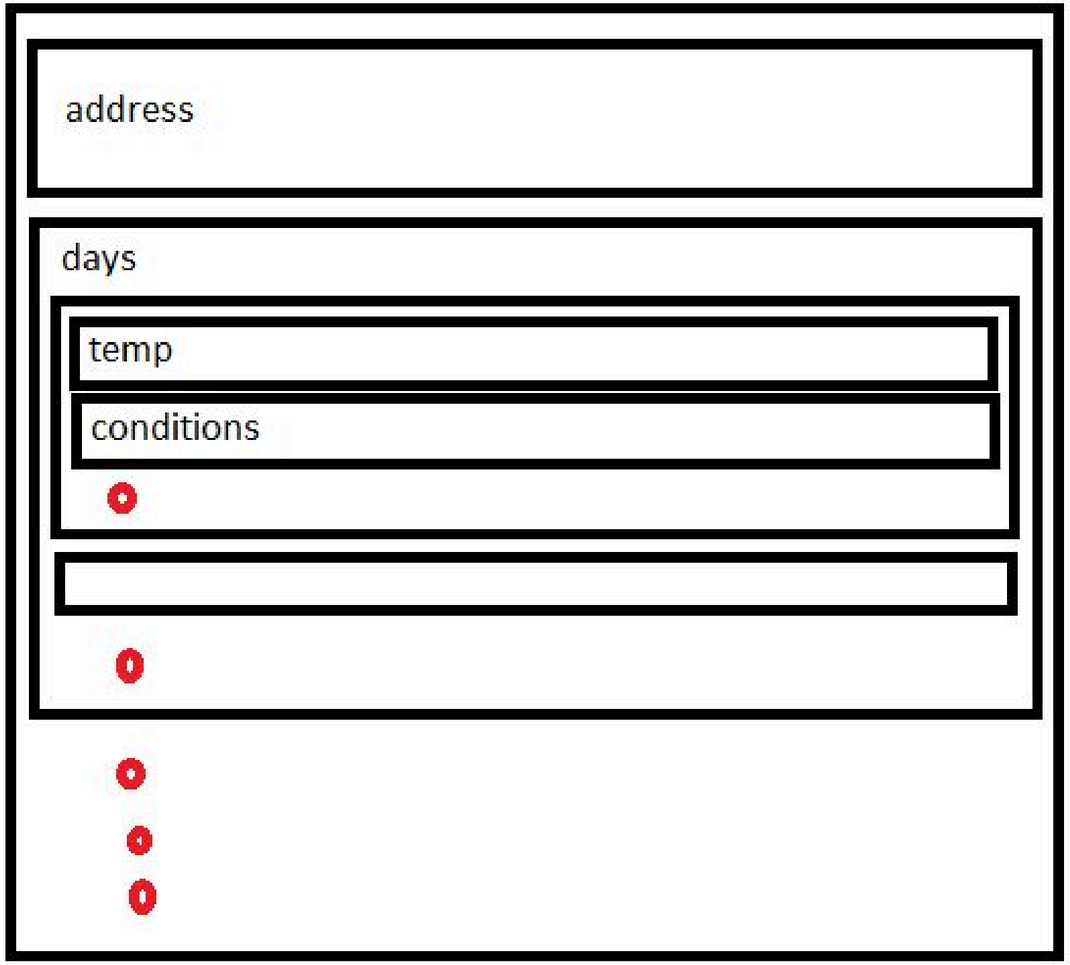
```
export const fetchRealWeather = async city => {

  // console.log("I am inside fetchRealWeather");

  const response = await fetch(
    `https://weather.visualcrossing.com/VisualCrossingWebServices/rest/
  );

  const jsonResponse = await response.json();

  // console.log("I am after the fetch call inside fetchRealWeather");

  // console.log(typeof jsonResponse.address);
  // console.log(typeof jsonResponse.days[0].conditions);
  // console.log(typeof jsonResponse.days[0].temp);

  return {
    location : jsonResponse.address,
    weather : jsonResponse.days[0].conditions,
    temparature : jsonResponse.days[0].temp
  };

}
```

Networking

- The approach we've taken so far is a common pattern used when building brand new React Native applications. We first organized our views using components and then introduced some state and state management.
- However, nobody will find our app useful unless it's actually connected to real data. When building a new mobile app, chances are we'll need to communicate with a server. Communicating with a server is a crucial component of most mobile applications.
- For the purpose of this application, we'll use the VisualCrossing API to fetch real weather information.
- We will parse our response in json. Json formats its data in objects.



The red color circle, synchronizing the request and response action.



Conditional Rendering

- Depending on the value of error we will show different output

```
{error && (
  <Text style={[styles.smallText, styles.textStyle]}>
    Could not Load weather, Please try different city{' '}
  </Text>
)}

{!error && (
  <View>
    <Text style={[styles.largeText, styles.textStyle]}>
      {location}
    </Text>
    <Text style={[styles.smallText, styles.textStyle]}>
      {weather}
    </Text>
    <Text style={[styles.largeText, styles.textStyle]}>
      {' '}
      {temperature}℃
    </Text>
  </View>
)}
```

Summary

- Built our very first React Native application.
- Built out each of the components that make up our application using the built-in components provided by React Native.
- Dived into the fundamentals of React Native understanding JSX, how to apply custom styling as well as understanding how to use props and state to manage and control data.
- Moved on to more complex topics including lifecycle methods and how to use external network calls to provide real content to our application.